T1: Problemas Clássicos de Concorrência

FPPD — Fundamentos de Processamento Paralelo e Distribuído (98713-04)
Grupo: João Pedro Bianchi, Gustavo Both, Antônio Costa — Repositório: https://github.com/dejotaa15/FPPD

Os dois programas foram escritos em Go usando apenas a biblioteca padrão. Todas as medições foram feitas com Go 1.27.0 em Windows 11 (amd64, 12 CPUs lógicas). Todas as versões foram executadas com `go run -race` sem nenhum aviso do detector. Os dados deste relatório são reproduzidos pelo script `experimentos.ps1`.

1. Jantar dos Filósofos

1.1 Solução e decisões de projeto

Cada garfo é um channel de capacidade 1 que contém uma ficha: pegar o garfo é receber a ficha (bloqueia se o garfo está em uso) e largar é devolvê-la. Cada filósofo é uma goroutine que repete pensar → adquirir garfos → comer → liberar. A forma de adquirir é definida pela interface `estrategia`, com quatro implementações escolhidas pelo parâmetro `-estrategia`:

- **base**: todos pegam o garfo da esquerda e depois o da direita.
- **hierarquia**: os garfos são ordenados pelo índice e todo filósofo pega primeiro o de menor índice. Na prática só o último filósofo inverte a ordem (pega o garfo 0 antes do N−1).
- **limite**: um channel de capacidade N−1 funciona como semáforo de contagem e permite que no máximo N−1 filósofos disputem garfos ao mesmo tempo.
- **garcom**: uma goroutine garçom recebe pedidos e devoluções por channels e só autoriza um filósofo quando os dois garfos dele estão livres. O estado da mesa pertence só ao garçom, sem mutex.

Entre pegar o primeiro e o segundo garfo há uma pausa (`-pausa`, 1 ms por padrão). Ela alarga a janela em que o filósofo segura um garfo e espera o outro, o que torna o deadlock da versão base reprodutível, e é aplicada a todas as estratégias para que sejam comparadas nas mesmas condições. Os tempos de pensar e comer são sorteados entre 0 e 1 ms. O programa recebe N e R por parâmetro e tem dois modos: no padrão cada filósofo faz R refeições (mede a espera); com `-disputa` as N·R refeições formam um estoque comum e cada filósofo come quantas conseguir (mede a distribuição).

1.2 Análise de deadlock e starvation

Versão	Deadlock	Starvation	Justificativa
base	ocorre	—	As quatro condições de Coffman valem ao mesmo tempo.
hierarquia	não ocorre	possível em teoria	Elimina a espera circular. Não tem mecanismo próprio contra starvation.
limite	não ocorre	possível em teoria	Elimina a espera circular. Não tem mecanismo próprio contra starvation.
garcom	não ocorre	não ocorre	Elimina a posse e espera. Fila com reserva para o pedido mais antigo.

Base. Exclusão mútua: cada garfo tem uma única ficha. Posse e espera: o filósofo segura o garfo esquerdo enquanto espera o direito. Não preempção: ninguém tira o garfo de outro, ele só é devolvido voluntariamente depois de comer. Espera circular: o filósofo i espera o garfo i+1, que está com o filósofo i+1. Com a pausa de 1 ms o deadlock ocorreu antes da primeira refeição em todas as execuções, e o programa imprimiu o ciclo (filósofo 0 segura o garfo 0 e espera o 1, ..., filósofo 4 segura o 4 e espera o 0). O deadlock depende do entrelaçamento: sem pausa e pensando até 1 ms, uma execução completou as 5000 refeições sem travar; sem pausa e sem tempos de pensar e comer, travou depois de 38 a 60 refeições por filósofo. Ou seja, a versão base pode funcionar por muito tempo e ainda assim está errada.

Hierarquia. Um ciclo de espera exigiria que algum filósofo segurasse um garfo de índice maior enquanto espera um de índice menor, o que a ordem global proíbe. As outras três condições continuam valendo. Starvation: a solução não garante nada por si só; ela depende da ordem em que o channel do garfo atende quem está bloqueado. O runtime do Go atende em ordem de chegada (FIFO), por isso nenhum filósofo deixou de comer nos testes, mas isso é um detalhe de implementação do runtime e não uma garantia da linguagem.

Limite. Com no máximo N−1 filósofos disputando N garfos, o ciclo de N filósofos necessário para a espera circular não se forma: sempre sobra um garfo para alguém completar o par. Sobre starvation vale a mesma observação da hierarquia, agora também para a fila do channel de lugares.

Garçom. O filósofo só toca nos garfos depois de autorizado, e a autorização só sai com os dois livres. Assim ninguém segura um garfo esperando outro: a condição de posse e espera é eliminada. Um garçom que apenas atendesse "quem pode comer agora" permitiria starvation: os dois vizinhos de um filósofo podem se revezar indefinidamente. Por isso o garçom mantém os pedidos em fila e o pedido mais antigo ainda não atendido reserva seus dois garfos, que não podem ser dados a pedidos mais novos. O pedido mais antigo é atendido assim que os vizinhos atuais terminam, e todo pedido acaba se tornando o mais antigo, então a espera é limitada independentemente do runtime.

1.3 Análise de fairness

Dados com N = 5 e R = 1000. A espera é o tempo entre o filósofo ficar com fome e ter os dois garfos, e inclui a pausa de 1 ms. O índice de Jain vale 1 quando todos comem a mesma quantidade e 1/N quando um só come tudo.

Tabela 1 — Espera média pelos garfos (µs), modo padrão (1000 refeições para cada filósofo).

Estratégia	F0	F1	F2	F3	F4	Média	Pior espera	Tempo total
hierarquia	3161,6	2011,5	1694,4	1450,4	3164,2	2296,4	9827,6	4,87 s
limite	2494,4	2496,8	2495,5	2511,1	2469,6	2493,5	8628,5	4,16 s
garcom	3540,0	3546,5	3540,0	3554,7	3529,4	3542,1	7701,6	5,25 s

Tabela 2 — Refeições por filósofo, modo `-disputa` (5000 refeições disputadas entre os cinco).

Estratégia	F0	F1	F2	F3	F4	Mín/Máx	Índice de Jain	Tempo total
hierarquia	764	1062	1147	1263	764	764 / 1263	0,9604	3,90 s
limite	1000	1000	1000	1000	1000	1000 / 1000	1,0000	4,20 s
garcom	998	1000	1002	997	1003	997 / 1003	1,0000	5,24 s

Hierarquia é a menos justa. Os filósofos 0 e 4 esperam mais que o dobro do filósofo 3 (3,16 ms contra 1,45 ms) e, em disputa, comem 40% menos (764 contra 1263). O resultado se repetiu em três execuções extras (mínimo entre 756 e 762, máximo entre 1240 e 1246, Jain entre 0,959 e 0,961). A causa é a própria quebra de simetria: o garfo 0 é o *primeiro* garfo tanto do filósofo 0 quanto do 4, então os dois disputam entre si logo na entrada. Já o garfo 4 é o *segundo* garfo dos filósofos 3 e 4, e por isso só é pedido por quem já venceu uma disputa, o que favorece o filósofo 3. Ninguém ficou em starvation, mas o acesso é desigual e a desigualdade depende da posição na mesa.

Limite é a mais uniforme. As esperas médias diferem em menos de 2% e, em disputa, cada filósofo fez exatamente 1000 refeições nas quatro execuções realizadas. Como a mesa está sempre saturada, os filósofos formam fila no channel de lugares, e a fila FIFO do runtime transforma o acesso em um rodízio.

Garçom é justa, porém a mais lenta. A distribuição é praticamente perfeita (997 a 1003) e a pior espera é a menor das três (7,7 ms), mas a espera média é cerca de 40% maior que a do limite e o tempo total é o maior. Nossa explicação é o custo da garantia: o garfo reservado para o pedido mais antigo fica parado mesmo que um vizinho pudesse usá-lo, e cada refeição passa por duas trocas de mensagem com o garçom. Em resumo, a hierarquia tem o menor custo e a pior fairness, o limite teve o melhor equilíbrio nos testes mas depende da fila do runtime, e o garçom é o único que garante ausência de starvation por construção, pagando com throughput.

1.4 Encerramento ordenado

A `main` espera os filósofos com um `WaitGroup`. Em paralelo, um vigia confere a cada 2 s o contador de refeições; se ele não mudou, declara deadlock, imprime o que cada filósofo segura e espera, e fecha o channel `abortar`. Toda espera por garfo é um `select` entre o garfo e `abortar`, então os filósofos bloqueados acordam, devolvem o garfo que seguram e terminam. Assim até a versão base encerra sem goroutines bloqueadas. O garçom é encerrado pelo channel `fim` e a `main` espera sua confirmação. Ao final o programa imprime `runtime.NumGoroutine()`, que foi 1 (só a `main`) em todas as execuções.

2. Produtor/Consumidor com Buffer Limitado

2.1 Solução e decisões de projeto

As duas versões implementam a mesma interface `Buffer` (`Inserir`, `Retirar` com limite de tempo, `Fechar`, `Ocupacao`), de modo que produtores, consumidores, medições e verificação são o mesmo código para ambas (`-versao channel|semaforo`).

- **Channel**: o buffer é um `chan int` de capacidade K. O bloqueio com buffer cheio ou vazio é a semântica do Go.
- **Semáforos**: buffer circular (vetor de K posições, índices `entrada` e `saida`). O semáforo de contagem é um tipo próprio construído sobre um channel de fichas, com operações `P` e `V`. `naoCheio` começa em K e conta posições livres; `naoVazio` começa em 0 e conta itens. Dois semáforos binários, `mutexEntrada` e `mutexSaida`, protegem os índices: sem eles, dois produtores leriam o mesmo `entrada` e escreveriam na mesma posição, e dois consumidores retirariam o mesmo item. Produtor: `P(naoCheio)`, `P(mutexEntrada)`, escreve, `V(mutexEntrada)`, `V(naoVazio)`; o consumidor é simétrico.

Cada item tem um identificador único (`produtor × itens + sequência`). Ao final o programa confere que o total consumido é igual ao produzido e que cada identificador foi consumido exatamente uma vez, o que detecta tanto perda quanto duplicação. A verificação passou em todas as execuções deste relatório.

2.2 Análise de deadlock e starvation

Channel. Não há deadlock: cada goroutine espera por um único recurso (espaço ou item) sem segurar outro, então não existe posse e espera. O único bloqueio permanente possível seria o de consumidores esperando em um buffer vazio depois do fim da produção, resolvido pelo encerramento (2.5). Starvation: a linguagem não garante ordem, mas o runtime atende os bloqueados em FIFO; no cenário A cada consumidor recebeu entre 245 mil e 255 mil dos 1 milhão de itens.

Semáforos. O produtor segura uma posição (ficha de `naoCheio`) enquanto espera `mutexEntrada`, então posse e espera existe, mas a espera circular não: quem está com um mutex nunca espera por mais nada, apenas escreve ou lê e libera. A ordem dos `P` é a clássica (contagem antes do mutex), para que ninguém durma segurando o mutex. Starvation: mesma observação do channel; cada consumidor recebeu entre 249 mil e 252 mil itens.

2.3 Efeito da capacidade K

P = 4 e C = 4 fixos, K ∈ {1, 10, 100}, mediana de 5 execuções. Cenário A: 1 milhão de itens sem trabalho simulado, mede o custo da sincronização. Cenário B: 2000 itens com tempos de produção e de consumo sorteados entre 0 e 2 ms, representa carga em rajadas. A ocupação é amostrada a cada 200 µs por uma goroutine monitora; "Inserir" e "Retirar" são o tempo médio que produtor e consumidor passam dentro da operação. Em todas as execuções os consumidores receberam quantidades próximas (no pior caso, 469 e 522 itens de 2000).

Tabela 3 — Cenário A (sem trabalho simulado, 1.000.000 de itens).

Versão	K	Throughput (itens/s)	Ocupação média	Inserir (µs)	Retirar (µs)
channel	1	1.150.187	0,57 (57%)	3,45	3,44
channel	10	1.707.837	6,46 (65%)	2,32	2,31
channel	100	2.442.988	74,19 (74%)	1,62	1,61
semáforo	1	584.445	1,00 (100%)	6,83	6,80
semáforo	10	677.163	9,48 (95%)	5,88	5,85
semáforo	100	686.305	99,26 (99%)	5,80	5,78

Tabela 4 — Cenário B (produção e consumo entre 0 e 2 ms, 2000 itens).

Versão	K	Throughput (itens/s)	Ocupação média	Ocupação máx.	Inserir (µs)	Retirar (µs)
channel	1	2739	0,48	1	116,96	149,11
channel	10	2912	4,30	10	14,45	53,04
channel	100	2987	16,13	26 a 63	0,00	27,32
semáforo	1	2735	0,51	1	123,80	141,89
semáforo	10	2929	3,97	10	10,80	61,44
semáforo	100	2997	14,53	24 a 50	0,00	23,03

K desacopla produtores de consumidores. Com K = 1 os dois lados andam em passo travado: cada inserção depende de uma retirada. No cenário B o produtor fica em média 117 µs bloqueado por item; com K = 10 esse tempo cai para 14 µs e com K = 100 vai a zero, porque a ocupação máxima observada (63) nunca chegou à capacidade. A partir daí os produtores não percebem mais os consumidores: o buffer absorve as rajadas. O ganho de throughput no cenário B é pequeno (cerca de 9% de K = 1 para K = 100) porque o tempo é dominado pelo trabalho simulado, não pela espera.

Quando a sincronização domina (cenário A), K maior reduz trocas de contexto. No channel o throughput mais que dobra de K = 1 para K = 100: com folga no buffer, uma goroutine faz várias operações seguidas sem ser suspensa. A ocupação média acima de 50% em todos os casos mostra que os produtores são um pouco mais rápidos que os consumidores, que ainda guardam cada item para a verificação.

Channel contra semáforos. No cenário A a versão com semáforos tem de metade a um quarto do throughput do channel e quase não melhora de K = 10 para K = 100. Cada operação dela faz quatro operações de channel (dois semáforos de contagem e o `P/V` do mutex) contra uma do channel nativo, e nossa hipótese para o ganho pequeno com K é que o gargalo passa a ser a disputa pelos mutexes, que não depende de K. As ocupações não são diretamente comparáveis: na versão com semáforos uma posição conta como ocupada desde a reserva pelo produtor até a liberação pelo consumidor, e o buffer fica praticamente cheio o tempo todo. No cenário B as duas versões são equivalentes (diferenças dentro da variação entre execuções), pois o custo de sincronização é desprezível frente ao trabalho de milissegundos.

2.4 Consumo com timeout

Todos os consumidores retiram com `select` e `time.After`. Na versão channel o `select` escolhe entre o channel de itens e o timer; na versão com semáforos, como o semáforo é um channel, o `P(naoVazio)` entra no mesmo `select`. Se o tempo esgota, o consumidor executa o comportamento alternativo: registra a ociosidade (contador, e mensagem com `-v`) e volta a tentar. Com produtores lentos (P = 2, C = 3, K = 4, 100 itens, produção de até 40 ms, timeout de 10 ms) foram 223 timeouts na versão channel e 203 na versão com semáforos, e os 100 itens foram consumidos nas duas. Nos cenários A e B, com timeout de 50 ms, não houve timeouts.

2.5 Encerramento ordenado

A `main` segue quatro passos: (1) `Wait` nos produtores; (2) `Fechar` o buffer; (3) `Wait` nos consumidores; (4) parar a goroutine monitora e esperar sua confirmação. Na versão channel, `Fechar` é `close`: o Go entrega todos os itens restantes antes de sinalizar o fechamento, então os consumidores drenam o buffer e saem. Na versão com semáforos, `Fechar` fecha o channel `encerrado`; o consumidor que o percebe tenta um `P(naoVazio)` sem bloquear e só sai se não houver ficha. Como todos os `V(naoVazio)` já aconteceram antes de `Fechar`, não haver ficha significa que todo item restante já foi reservado por outro consumidor, e nada se perde. O programa termina com uma goroutine ativa (a `main`) e com código de saída diferente de zero se a verificação falhar.

3. Uso de ferramentas de IA

Foi utilizado o **Claude** (Anthropic), pela ferramenta Claude Code, em todas as etapas do trabalho:

- **Projeto da solução**: proposta da estrutura dos dois programas (interface de estratégias, garfos como channels com ficha, interface comum de buffer, semáforo construído sobre channel) e das métricas coletadas.
- **Geração de código**: o código-fonte dos dois programas e o script de experimentos foram gerados pela ferramenta.
- **Depuração**: execução com `-race` e ajustes, como a política de reserva do garçom (a primeira versão reservava garfos para todos os pedidos da fila e praticamente serializava as refeições) e a definição de ocupação na versão com semáforos.
- **Análise dos dados e redação**: execução dos experimentos, montagem das tabelas e primeira versão deste relatório.

O grupo executou os programas, conferiu os resultados e estudou o código e as análises entregues, de modo que cada integrante é capaz de explicar qualquer parte do trabalho.